1 - Fundamentals

Pseudocode

Good pseudocode is algorithmic, not language specific, simple, and easily human readable.

Algorithmic

An algorithm is like a recipe. Defining an algorithm should prescribe the ingredients (inputs), the results (outputs), and the steps (the stepwise, ordered process) necessary to successfully complete the task. Algorithmic development is not simply implementing the process but it is also clearly documenting the algorithm including documenting any and all inputs and any and all outputs.

Pseudocode should be organized as a stepwise sequence and not as a paragraph. Pseudocode is used for organization and logical analysis at the beginning of algorithmic development, but it also can act as key documentation of the implementation as it allows a stakeholder to understand the process without needing to read syntactically detailed code. In other words, the pseudocode that a programmer may write to initially understand and organize the logic of a process can become the documentation embedded in the code which can be critical for future debugging or further development. In this regard, writing pseudocode that is too symbolic and too close to the programming language becomes redundant and does not add any information to the code. Consider the following example where pseudocode is written using Java comments...

```
// for i = 0 to n-1
for(int i = 0; i < n; i++) {
    // print A[i]
    print( A[i] );
}
```

The above pseudocode meets general pseudocode criteria, but it is redundant and does not really add any information. It is actually more distracting than useful because it provides no new information and any stakeholder still has to infer the intent of the code. Consider this alternative:

```
// iterate over all elements in A
for(int i = 0; i < n; i++) {
    // print each element to the console
    print( A[i] );
}
```

In this example, the pseudocode provides additional context and does not require someone to interpret the code to understand what the process is doing. If we omit the implementation it would leave the following...

```
// iterate over all elements in A
    // print each element to the console
```

The above is probably an artifact of the first algorithmic developmental step where the developer reasoned out the process in human terms. The added code was simply a translation step into the implementation language from the pseudocode.

We can perform time complexity analysis on pseudocode before investing the time to write syntactically correct and debugged in depth code which means we can assess whether it is a correct approach before spending costly, limited resources on an implementation.

Not Language Specific

As suggested, good pseudocode is not required to conform to any language's specific syntax. Good quality pseudocode should be easily translatable to any language but there is no single or formal definition of pseudocode. Programmers need to develop their own pseudocoding skill and also the flexibility to read other programmers pseudocode. Good pseudocode should be both simple and easily human readable.

Code in a specific language must adhere to syntactic and allocation rules and can become deeply symbolic and dependent on evolving steps. Pseudocode frees a programmer from focusing on language specific rules and lets a programmer focus on the process and quickly sketch it out without the need to debug language specific details. A programmer should work out the logic of an algorithm first and then focus on translating it into a specific language. This allows a programmer to first debug the logic in the abstract language so the programmer may know it is algorithmically correct. Once a working model is proposed, the programmer can then focus on translating it according to the specific syntactic nuances of a the implementation language.

Simple

By keeping pseudocode simple, it can omit trivial details that are just not necessary to communicate the idea. For example, it is a common pattern to use a for loop with an index that increments the index each pass through the loop to complete some repetitive task. Unless there is something very special about the incrementing of the index, it is not necessary describe incrementing the index. In fact, pseudocode for a loop might completely abstract away the specifics of the for loop and focus on the overall task. Just as in the example above:

```
// iterate over all elements in A
```

suggests a loop without prescribing the implementation. A language may not even have a `for` keyword, so using more generalized language might be more easily translatable. In the example, there are no indices and no for loop, yet the idea of looping is fully described.

There are some associated costs like, the creation of space in memory in an implementation, but these details often distract from and are superfluous to the algorithmic analysis. Yes allocating space is costly in terms of both time and space, but it is generally linear time at worst and most algorithms are linear time at best, so including pseudocode about allocation, unless it is the central feature of the algorithm, is distracting and mostly irrelevant to the more important exercise of sketching out the logical steps involved with an algorithm. Memory allocation is also usually closely associated with language specific features. For this reason, most memory management will not be mentioned in the pseudocode sketch of an algorithm unless it is a critical feature of the algorithm.

Easily Human Readable

We write pseudocode for stakeholders not for the computer. In other words, we write pseudocode for people and not for automated systems.

One of the purposes of AI systems is to bring computing closer to human communication and reasoning, so we would prefer to provide more human oriented prompts or sketches to the computer. In this regard it would benefit the novice programmer to learn how to more clearly communicate the ideas and structure of a process in the necessary logical order to generate code faster through these systems than to focus on writing perfect syntax in the first programming pass.

Reading code is non-productive time, or in other words, no one gets paid to read code. Over the lifetime of the program, someone will likely have to read through code you write, most likely you. Debugging is costly in terms of time and money. We do not want to break working code. Changes are costly. If pseudocode is written before implementation it can be used as comments for development and it can create a scaffold for code design that allows the programmer to fill in the blanks. It is considered good practice to sketch out the process as pseudocode comments first and then write the actual code in between each pseudocode step. This also has the added benefit that it leaves clear explanations that summarize segments of code so that debugging is safer and easier.

Activity Times

Java is a compiled language. In other words, there are three activity times for development involving a compiled language:

1. Design Time
2. Compile Time
3. Run Time

Design Time

Design time encompasses all the activities around writing code. Design time is the time involved with writing code. Students often wrongfully associate actions with design time. The central issue revolves around that nothing really happens during design time and students associated design time with actions like allocating memory; however, one key is that allocation can only happen during execution,

Compile Time

Compile time encompasses all the activities around compiling code. This activity is mostly out of the hands of the programmer as this is the step where a rather sophisticated program interprets the code produced during design time. This is a translation step by very definition as the compiler is translating the steps defined by a high level language into the more primitive instructions necessary to fulfill the high level design. One of the mistakes of a novice programmer is to imagine that small structural changes at the design level affect the overall system generated by the compiler. The fact of the matter is that the compiler makes decisions about the code provided to the compiler that are difficult to anticipate and therefore small, deeply debated changes at design time can have no impact on the code produced by the compiler. Optimizing design time code such that the code produced by the compiler requires extensive understanding of the compiler. For the most part, the novice programmer

hallucinates that changes to the code will produce any marketable improvement in the program actually produced. In other words, without extensive knowledge of the compiler, modest code restructuring will likely have no impact on the run time efficiency of a program.

Run Time

Run-time encompasses all activities around running code. Students again hallucinate a number of activities at design time that are actually performed at run time. For example, a variable is never allocated until a program actually runs. Students tend to deeply associate allocation of a variable with design time activities when in fact allocation is a run time operation. In other words, there is no memory available and therefore no memory can be allocated at design time;

however, at run time, we cannot actually execute unless memory is available.

State

The term **state** captures the idea of the values of a system at a specific point in time. What can we generally say about `x` and `i` in the following example:

```
x = 0;
for( int i = 0; i < 10; i++ ) {
    x = x + i;
}
```

Variables are a programming tool because a program must change values throughout its execution. The term variable literally means it has the capacity to change over time, so it is important for a programmer to consider the state of a variable with respect to a specific point of time and to rarely assume the state of a variable at any point in time.

We can loosely associate vertical position in adjacent regions of the program with time. Unless there are some control elements that change the flow of execution, lines in the same function will be processed from top to bottom. For this reason, line numbers become a loose proxy for time, but it is again important to be clear about the the control elements around a specific line number. If a specific line is inside a loop, a programmer needs to not only consider the line number but also the iteration of the loop.

Generally, with each line executed, there is some change to at least one value in the overall system. As the program proceeds through lines, it is “evolving”. In other words the value of variables are changing with respect to time. We not only use the term state to describe the snapshot of a variable at a specific line but also the snapshot of an entire system with respect to a specific point in time. For example:

```
x = 0;
for( int i = 0; i < 10; i++ ) {
    x = x + i;
}
```

We may concern ourselves with the state of `i` or with the state of `x` as independent variables, or we may concern ourselves with the state of the entire system including both `i` and `x` as a snapshot of the entire system at any specific point in time. In other words, state may refer to the "status" of a single variable, of an object, or of an entire system, but it implicitly means that we are capturing this information at a specific point in time as it will change with further evolution of the system

When debugging, we have to be very familiar with and careful of assessing the state of the system. A bug arises because the system enters an unanticipated state that the system was not designed for. When discussing state, it is important to specify both the scope of what is being assessed, variable, function, object, system, and the time at which it is being measured.

We may **trace** a system. Tracing means to track the evolution of a system by hand to identify where the state is invalid.

Java Types

Java is a **strongly typed language** which means that all variables have to be declared before they can be used. Every Java variable **declaration** has two required components: `<type>` and `<label>` in other words a Java variable declaration follows the form:

`<type> <label>;`

For example, each of the following are examples of a Java declaration:

```
boolean success;
int i;
float x;
int[] A;
String msg;
```

In each of the above examples, a `<label>` is declared with a specific `<type>`. In Java, and any strongly typed language like C, the label cannot appear before the declaration, so the declaration must always be made "above" the usage of any symbol using the declared label. Considering that programs are generally processed from top to bottom, before means that any variable must be declared at a line above the symbol's usage in any expression. In other words, the symbol's type needs to be clearly specified before it is used in any way.

The purpose of a declaration is to notify a language's compiler of usage and syntactical rules for a specific variable. This is a feature and while it may sometimes be in the way for a language like Java, it is one of the purposes of using Java. Consider that in Python, variables do not need to be declared and so illegal operations may only be discovered at the time of execution. For a strongly typed language, some operations can be validated at the compile time simply because of type declaration. The reasoning here is that one signifier of correct logic is following consistent syntactical rules, and if syntactical rules do not exist or are loosely enforced, then there is a higher likelihood of errors in the program. In other words, one of the services a strongly typed language can perform is to indicate errors that arise due to a program violating syntactic rules. These types of errors can be detected before run time during compile time..

Meta-type

In Java, the `<type>` declared must be one of the available and legally defined types. All types can be broadly classified as belonging to one of two meta-types.

A **primitive type** represents the basic data built into a programming language. There are eight Java primitive types each of which is defined by a specific Java keyword and commonly color coded in modern IDE's to easily identify them. The Java primitives are byte, short, int, long, float, double, boolean, and char. All other types including arrays and types defined through classes are non-primitive and are represented as a **reference type**. Variables are either primitive or non-primitive. If a variable is declared with one of the eight primitive types, it must be primitive, and if a variable is not one of the eight primitive types, it must be non-primitive and therefore a reference type.

Primitives:

```
boolean success;
int i;
float x;
```

Non-primitives are everything else:

```
int[] A;
String msg;
MyCustomClassType o;
```

We make a distinction between primitives and non-primitives because primitives are simple, fixed size, single values in memory while non-primitives are containers that may hold sets of values. Since primitives are fixed in size, they are stored in a single location in memory, and since non-primitives may be composed of multiple values, they need a range of memory to store them. Because each of these two metatypes have such different memory requirements and behaviors, they are treated very differently in code.

The value stored in a non-primitive is really a memory address. This address references where the container exists in memory which means the value stored in memory for all non-primitive types is interpreted as a location. Since there is an indirect relationship between the variable value and the container for non-primitive types, we may describe non-primitive variables as "reference types", *i.e.* they refer to the location of their data, or "pointers", *i.e.* they point to the location of their data. Because a reference type has two pieces: the variable and the object it refers/points to, both pieces have to be managed.

Consider the following declaration and what it instructs the compiler to do.

```
int[] A;
```

A programmer will typically describe the above as declaring an integer array; however, this is not technically correct. The declaration actually instructs the compiler to set aside a pointer A that is allowed to point to an integer array and it does not actually instruct the compiler to create an array. Creating the array requires an additional step and the use of the new keyword.

```
A = new int[5];
```

This second step is what makes the request to the compiler to allocate an array with space for 5 integers. Whenever working with non-primitives, we must be mindful that creating the reference is a separate step from creating the object and that it is legal for a reference to refer to nothing which we call `null`. Primitive types always must contain a legal value, so they do not point and they can never be nothing.

Initialization

Another concern for variables is establishing initial state. Optionally, a Java variable declaration may assign a specified `<initial value>` to the variable overriding assignment of the default value or a programmer may allow Java to assign a default value for `<initial value>` if one is not specified by the programmer. The `<initial value>` must be a value that conforms to the declared type. A Java declaration that assigns an initial value follows the form:

```
<type> <label> = <initial value>;
```

The following are examples of appropriate values for each of the associated types `<initial value>`:

```
boolean success = false;
int i = 0;
float x = 1.0;
String msg = “”;
```

All data in memory is a number whether it is a `byte`, `boolean`, `char`, or even non-primitive type. The variable type instructs Java how to interpret the underlying numeric representation in memory.

A `boolean` is interpreted as either `true` or `false`. If the numeric value stored in the `boolean` variable's memory location is location zero, the `boolean` variable is interpreted to have the value of `false`. If the numeric value stored in the `boolean` variable's memory location is non-zero, the `boolean` variable is interpreted to have the value of `true`.

Numeric types have specific formats but generally a zero representation in these formats is interpreted as a zero numeric value.

Non-primitives are really addresses of locations in memory, so a zero value is interpreted as referencing the zero address in memory. The zero address has a special role on a computer and it is generally illegal for any program to access the zero address. As such the zero address has a special meaning of representing nothing and the special keyword of `null` to identify it and suggest it is nothing.

If no initialization is provided with a Java declaration, Java automatically assigns the variable a default value of zero to memory and the type determines how that value is interpreted.

```
boolean success;    // zero boolean is interpreted as false
int i;              // zero int is interpreted as 0
float x;            // zero float is interpreted as 0.0
String msg;         // zero non-primitive is interpreted as null
```

The above is equivalent to:

```
boolean success = false;
int i = 0;
float x = 0.0;
String msg = null;
```

Non-primitive reference types are either assigned a `null` pointer, *i.e.* they don't point to a legal address yet, assigned to point to a newly constructed object as signaled with the `new` keyword, or assigned to point to an already existing object through another variable.

Proof By Exhaustion

There are a variety of strategies we can use to find an answer but one approach is to simply try every possibility. In other words, this strategy attempts to prove a hypothesis by exhausting all possibilities and it is known as "proof by exhaustion" or a "brute force approach".

Consider search for a moment. If our search strategy is to simply check everywhere, we are using a brute force approach. However, if our strategy is to order the set and then compare the middle value with our search value allowing us to eliminate either the first half or the second half from subsequent comparison, we are using another more refined strategy, specifically we call this strategy "divide and conquer".

Brute force is the most costly approach we might use other than some random approach. People naturally learn this in their lives and create their own strategies to minimize search. For example, it is a common strategy to avoid searching for our keys if we put them in the same location every day.

Exponentials and Logarithms

At some point in your mathematical education you learned exponentials (powers) and their relationship to logarithms. We will find that both of these mathematical functions will be useful from an analytical standpoint, so it is important to refresh our understanding of them as we begin this course. One bit of reassurance is that we will exclusively focus on exponentials and logarithms in base 2 for this course.

Base 2 Exponential

A base 2 exponential is expressed by the following formula:

$y = 2^x$

Since this is a base 2 exponential, every time x increases by 1. the output doubles the result of the preceding value of x. For example, if $x = 2$ the result $y = 4$ but if $x = 3$ then $y = 8$. The result y of the second case is exactly double the result of the first case but with only an increase in 1 for the value of x. This doubling is the consequence of the the choice of base and since the base equals 2 every increase in the exponent by 1 doubles

the previous result. Functions that involve a similar rate of change are typically referred to as having "exponential" growth.

If my investment grows exponentially year over year, I will grow quite wealthy in a short period of time. We can think of the initial equation as asking the question how much larger will be my investment after 5 years, $x = 5$, if it doubles year over year and the answer will be 32. In other words, if I invested $100 in year zero, my investment should be worth $3200 after 5 years. Exponentials can be powerful functions in terms of growth.

$2^0 = 1$	
$2^1 = 2$	$2^9 = 512$
$2^2 = 4$	$2^{10} = 1024$
$2^3 = 8$	$2^{11} = 2048$
$2^4 = 16$	$2^{12} = 4096$
$2^5 = 32$	$2^{13} = 8192$
$2^6 = 64$	$2^{14} = 16384$
$2^7 = 128$	$2^{15} = 32768$
$2^8 = 256$	$2^{16} = 65536$

Base 2 Logarithm

Logarithms are the “inverse exponential” and represent decay rather than growth. To understand the relationship of logarithms with the exponential recall the initial exponential equation:

$$y = 2^x$$

We can take the logarithm of both sides of this equation:

$$\log_2 y = \log_2 2^x$$

We take the base 2 logarithm of both sides because the exponential base is 2. Our goal is solve for x which means we need to eliminate the base on the right side of the equation. By taking the base 2 logarithm of the right side, the base of 2 in the exponent is cancelled. In other words, the equation becomes the following:

$$\log_2 y = x$$

Let's slightly rearrange the terms by swapping the terms on either side of the equal sign:

$$x = \log_2 y$$

What types of questions does this equation answer? The original question addressed the question of "if my money doubles every year, how much money will I have in x years". This new question is capable of addressing the question of "if my money doubles every year, how many years will it take for me to earn y". Similarly it can answer how many times can I divide this set in half until I have one element left.

$\log_2 1 = 0$	
$\log_2 2 = 1$	$\log_2 512 = 9$
$\log_2 4 = 2$	$\log_2 1024 = 10$
$\log_2 8 = 3$	$\log_2 2048 = 11$
$\log_2 16 = 4$	$\log_2 4096 = 12$
$\log_2 32 = 5$	$\log_2 8192 = 13$
$\log_2 64 = 6$	$\log_2 16384 = 14$
$\log_2 128 = 7$	$\log_2 32768 = 15$
$\log_2 256 = 8$	$\log_2 65536 = 16$

Review

1. What is an algorithm?
2. Compute the powers of two from 2^0 to 2^8.
3. Explain why the base 2 logarithm predicts the maximum number of times a discrete set can be repeatedly subdivided in half.
4. With diminishing rainfall over the years, water reservoirs are stressed throughout the country. Imagine a scenario where half the preceding years rainfall is collected each year. If the local reservoir collected 50 feet of water this year, in how many years will the collection be less than 1 foot of water? Model your answer with a mathematical equation. *Hint : approximate numbers are good enough.*
5. You are given a 416 card deck (eight 52 card decks) of playing cards for blackjack. You divide the deck precisely in half resulting in two decks of 208 cards. You repeat this process again on each of the generated decks which results in you having four decks of 104 cards. Model this process with a simple mathematical expression and use that model to predict the maximum number of times you can repeat this process before ending up with decks consisting of either 1 or zero cards.
6. You are given a deck of 26 cards each with a different letter of the English alphabet on it. You divide the deck in half resulting in two decks of 13 cards each. You repeat this process resulting in four decks containing either 6 or 7 cards. If you continue to repeat this process, eventually you will end when you have a number of decks consisting of either 0 or 1 cards.

   a. Model this problem with a diagram that illustrates the number of times this process is repeated. One step in the process is considered subdividing all of the available decks into smaller decks.

   b. Model this problem with a simple mathematical formula.
7. You have found a crypto coin that is projected to double in value year over year for the next seven years. If the coin is currently worth $0.20, the projection predicts it to be worth $0.40 next year and $0.80 the year after that, and so forth. Model the expected growth for this coin through seven years using either a diagram or mathematical formula.

8. Java is a strongly typed language, what does this mean?
9. What two minimum pieces of information are required for every Java variable declaration?
10. Give an example of a Java variable declaration.
11. What is the default value assigned by Java to every variable if an initial value is not explicitly provided?
12. Explain how Java handles assigning default values to variables if an initial value is not explicitly specified?
13. A memory location contains the value 100. If the value at that location is interpreted as a boolean, what value will be produced?
14. A memory location contains the value 0. If the value at that location is interpreted as a non-primitive, what value will be produced?
15. All Java types belong to one of two meta-types. What are the two "meta-types"?
16. The Java `int` type belongs to which "meta-type"?
17. Given the following Java variable declaration: `int[] x;`
    a. What type is being declared?
    b. What meta-type does the above type belong to?
18. What meta-type does the Java String type belong to?
19. Give two examples of a primitive type.
20. Give two examples of a non-primitive or reference type.
21. All non-primitive variables contain the same kind of data. How is the value stored in a non-primitive type interpreted?
22. What is meant by the term state?
23. What is the relationship between the status of a variable or system with respect to time?
24. Explain what is meant by "the state of the program at line 5"?
25. What is the relationship between time and the values of variables in a program?
26. What does it mean to say that a program evolves through its execution?
27. Assuming an infinite universe, why would it be impossible to prove that life does NOT exist elsewhere in the universe?
28. What does it mean to use a "proof by exhaustion" or "brute force" strategy?
29. Recall our simple computer that can only store, add , negate, and conditionally loop.
    a. Why does the number of operations remain constant for the subtract function?
    b. Why does the number of operations in the multiply function vary with the size of the parameters?

Vocabulary

algorithm

pseudo-code

activity time

design time

compile time

run time

declaration

state

trace

type

meta-type

primitive type

reference type

brute force